In [1]:
```
import numpy as np
```

In [4]:
```
# Create a 1D array
a = np.arange(6)
print(a)
```

```
[0 1 2 3 4 5]
```

In [5]:
```
a2 = a[np.newaxis, :]
print(a2)
```

```
[[0 1 2 3 4 5]]
```

In [6]:
```
print(a2.shape)
```

```
(1, 6)
```

In [ ]:
```
a[np.newaxis, :]   # Same as
a[None, :]

a[:, np.newaxis]   # Same as
a[:, None]
```

A tuple is an ordered, immutable collection of items in Python.

Ordered means the elements keep their position. Immutable means you cannot change, add, or remove elements after the tuple is created. A tuple can contain different data types, such as integers, floats, strings, and even other tuples.

In [10]:
```
# Syntax
my_tuple = (1, 2, 3, 4, 5)
```

In [11]:
```
# Tuple of integers

numbers = (10, 20, 30, 40)
print(numbers)
```

```
(10, 20, 30, 40)
```

In [ ]:
```
a = np.array ([5,5,5,5,5,5])
print (a)
```

Out[ ]:
```
array([5, 5, 5, 5, 5, 5])
```

In [ ]:
```
# Tuple with different data types

student = ("Ali", 21, 3.75, True)
print(student)
```

```
('Ali', 21, 3.75, True)
```

In [12]:
```
# Single-element tuple
```

```
single = (5,)
print(type(single))
```

```
<class 'tuple'>
```

In [13]:
```
# Accessing elements
numbers = (10, 20, 30, 40)

print(numbers[0])    # First element
print(numbers[-1])   # Last element
```

```
10
40
```

Important properties of tuples

1. Ordered
2. Immutable (cannot be modified)
3. Can store duplicate values
4. Can contain different data types
5. Faster than lists for read-only data

Tuple vs List

1. Feature Tuple List
2. Syntax ( ) [ ]
3. Mutable ❌ No ✅ Yes
4. Ordered ✅ Yes ✅ Yes
5. Allows duplicates ✅ Yes ✅ Yes

In [14]:
```
numbers = (10, 20, 30)

# This will cause an error
numbers[0] = 100
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[14], line 4
      1 numbers = (10, 20, 30)
      3 # This will cause an error
----> 4 numbers[0] = 100

TypeError: 'tuple' object does not support item assignment
```

In [ ]:
```
import numpy as np

a = np.array([[1, 2, 3],
              [4, 5, 6]])

print(a.shape)

# Here, (2, 3) is a tuple of integers representing the array's shape: 2 rows and 3
```

```
(2, 3)
```

In [16]:
```python
# 1: Two 1D arrays
import numpy as np

# Create two arrays
a = np.array([1, 2, 3, 4, 5])
b = np.array([6, 7, 8, 9, 10])

print("Array A:", a)
print("Array B:", b)
```

```
Array A: [1 2 3 4 5]
Array B: [ 6  7  8  9 10]
```

In [17]:
```python
# 2: Two 2D arrays
import numpy as np

a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

b = np.array([
    [7, 8, 9],
    [10, 11, 12]
])

print("Array A:")
print(a)

print("\nArray B:")
print(b)
```

```
Array A:
[[1 2 3]
 [4 5 6]]

Array B:
[[ 7  8  9]
 [10 11 12]]
```

In [25]:
```python
# 3: Using np.arange()
import numpy as np

f = np.arange(6) # range of elements
a = np.arange(1, 6) # specific rang of elements
b = np.arange(2, 20)
g = np.linspace(0,10,num=5)
print (f)
print(a)
print(b)
print(g)
```

```
[0 1 2 3 4 5]
[1 2 3 4 5]
[ 2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19]
[ 0.   2.5  5.   7.5 10. ]
```

In [ ]: